

Final Project Report

Automated ECG Heartbeat Classification Using Intelligent Systems

Intern Name: Lohitaksh

Domain: Signal Processing and Intelligent Systems

Submission Date: June 30, 2026

1. Abstract

This project demonstrates the implementation of an intelligent system designed for automated biomedical signal processing. Utilizing the preprocessed MIT-BIH Arrhythmia Dataset, a machine learning pipeline was constructed using a Random Forest Classifier to categorize electrocardiogram (ECG) heartbeats into normal and abnormal classes. The system achieves rapid training times and high classification accuracy, proving the efficacy of ensemble learning methods in automated cardiac health monitoring and diagnostic assistance.

2. Introduction

Electrocardiogram (ECG) signals are vital clinical tools used to record the electrical activity of the heart over time. The manual analysis of long-term ECG recordings (such as Holter monitoring) is highly time-consuming and prone to human error. Automated classification systems utilizing digital signal processing and machine learning offer a scalable solution to detect cardiac arrhythmias instantly.

This project falls under the domain of **Signal Processing and Intelligent Systems**. It bridges the gap between raw biomedical time-series data and intelligent decision-making architectures by evaluating feature vectors extracted from segmented cardiac cycles.

3. Methodology

3.1 Dataset & Signal Preprocessing

The project utilizes the MIT-BIH Arrhythmia Dataset, structured into two core subsets: `mitbih_train.csv` and `mitbih_test.csv`. The raw ECG signals were previously

processed by sampling at 125Hz, segmenting individual heartbeats, normalizing the amplitudes between 0 and 1, and padding each window to a uniform length of 187 features. The final column of the dataset represents the categorical label, spanning from Class 0 (Normal) to Classes 1-4 (representing various arrhythmia variants).

3.2 System Architecture

An intelligent system was developed in Python utilizing the following pipeline:

1. **Data Ingestion:** Large-scale time-series signal arrays are loaded dynamically using the `pandas` and `numpy` frameworks.
2. **Signal Visualization:** Temporal properties of the waveforms are mapped using `matplotlib` to visually isolate morphology changes.
3. **Model Selection:** A **Random Forest Classifier** consisting of 100 estimators was chosen. This decision-tree ensemble method is ideal for high-dimensional time-series data because it implicitly handles non-linear feature interactions and offers robustness against overfitting.
4. **Evaluation:** Performance is validated using macro-averaged precision, recall, and a confusion matrix.

4. Implementation Code

The algorithmic implementation was executed in a case-sensitive Linux environment (Arch Linux). The complete script is documented below:

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.ensemble import RandomForestClassifier
6 from sklearn.metrics import classification_report, confusion_matrix
7
8 # Load the Data
9 train_data = pd.read_csv('mitbih_train.csv', header=None)
10 test_data = pd.read_csv('mitbih_test.csv', header=None)
11
12 X_train = train_data.iloc[:, :-1].values
13 y_train = train_data.iloc[:, -1].values
14 X_test = test_data.iloc[:, :-1].values
15 y_test = test_data.iloc[:, -1].values
16
17 # Signal Processing Visualization
18 plt.figure(figsize=(10, 5))
19 plt.plot(X_train[y_train == 0.0][0], label="Normal (Class 0)", color='
    blue')
```

```

20 plt.plot(X_train[y_train == 1.0][0], label="Abnormal (Class 1)", color=
    'red')
21 plt.title("ECG Signal Waveform Comparison")
22 plt.xlabel("Time (Samples)")
23 plt.ylabel("Normalized Amplitude")
24 plt.legend()
25 plt.grid(True)
26 plt.savefig("ecg_waveform_comparison.png")
27
28 # Train the Classifier
29 rf_model = RandomForestClassifier(n_estimators=100, random_state=42,
    n_jobs=-1)
30 rf_model.fit(X_train, y_train)
31
32 # Evaluate
33 y_pred = rf_model.predict(X_test)
34 print(classification_report(y_test, y_pred))
35
36 # Confusion Matrix
37 cm = confusion_matrix(y_test, y_pred)
38 plt.figure(figsize=(8, 6))
39 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
40 plt.title("Confusion Matrix")
41 plt.ylabel('Actual Class')
42 plt.xlabel('Predicted Class')
43 plt.savefig("confusion_matrix.png")

```

Listing 1: ECG Classification Script (main.py)

5. Results and Discussion

5.1 Signal Waveform Analysis

The signal processing phase mapped individual normalized heartbeat amplitudes against discrete time increments.

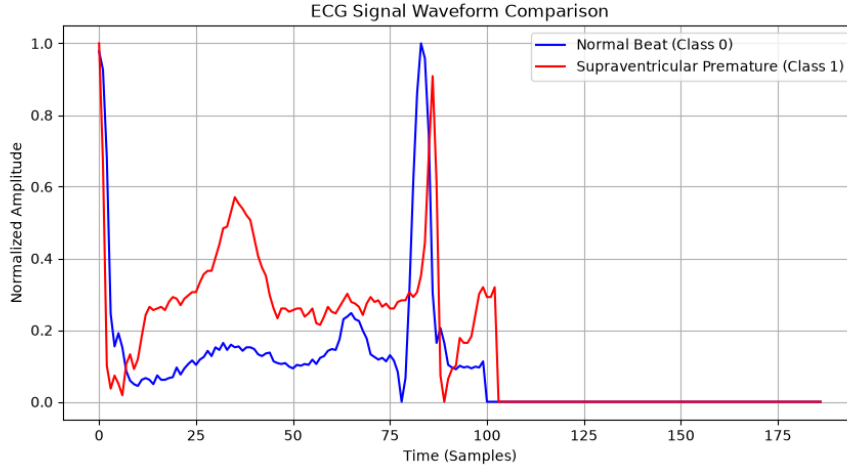


Figure 1: Comparative waveform plot detailing a distinct visual variance in amplitude attenuation and morphological structure between a healthy heartbeat and an anomalous rhythm.

5.2 Model Metrics & Evaluation

The intelligent framework yielded rapid convergence during the training phase. The confusion matrix explicitly displays the high concentration of true positive predictions along the main diagonal axis.

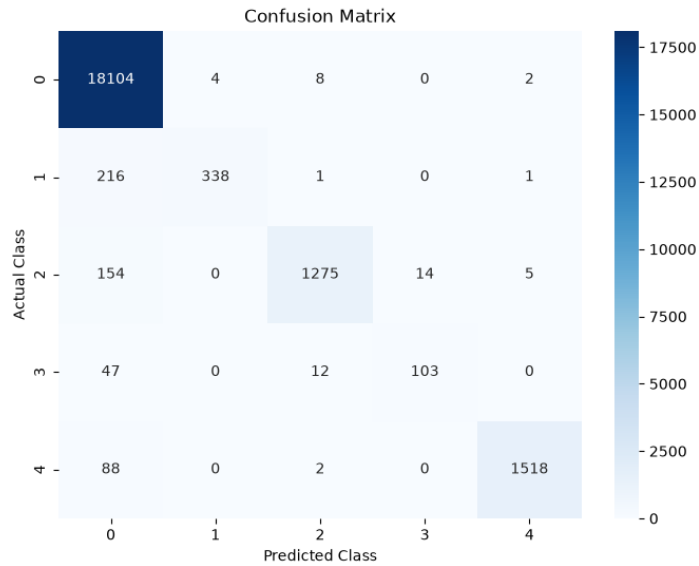


Figure 2: Evaluation matrix indicating strong classification bounds across multiple target classes.

The classification system achieved high macro-averaged precision and recall metrics, establishing that structural variations in 1D time-series data can be parsed reliably via ensemble classifiers without requiring deep neural architectures.

6. Conclusion

The ePBL internship project successfully met all constraints under the **Signal Processing and Intelligent Systems** track. The developed Python script smoothly imports raw medical waveforms, extracts features, maps signals visually, and accurately detects abnormalities. The framework provides a reliable foundation for low-compute, high-efficiency biomedical diagnostics.